

MindEase — Complete Technical Questions & Answers

Comprehensive technical documentation covering Database, APIs, Authentication, ML Model, Sensors, and Architecture

Section 1: Database

Q1. Which database is used and why?

A: MongoDB Atlas (cloud-hosted NoSQL) connected via `pymongo`. Chosen for schema-flexible documents that suit variable health scan data, native JSON support matching REST API responses, and free-tier cloud hosting via Atlas.

Q2. What are the database collections and their schemas?

Collection	Fields	Purpose
users	<code>_id</code> , name, email, password (bcrypt hash)	Registered user accounts
pending_users	name, email, password (hash), otp_code, created_at	Temporary storage during OTP verification
history	user_id, input (full StressInput dict), prediction_value (0/1/2), stress_level, timestamp	Stress scan results per user

All collections live in the database named `stress_db`.

Q3. How is the database connection established?

```
from pymongo import MongoClient
MONGO_URL = os.getenv("MONGO_URL")
client = MongoClient(MONGO_URL)
db = client["stress_db"]
users_collection = db["users"]
history_collection = db["history"]
pending_users_collection = db["pending_users"]
```

Q4. How is user data linked to their history?

A: The `user_id` field (string of MongoDB ObjectId) links users to scan history. On scan submission, backend stores `user_id` in the history document. Android app passes `user_id` from local `DataStore`.

Q5. How are timestamps handled in history records?

A: Dual strategy: Backend stores `datetime.utcnow()` for new records. Legacy records extract time from MongoDB `ObjectId.generation_time`. Android saves timestamps locally in `SharedPreferences`, matching them to backend records via a "scan key" hash (`blinkRate_heartRate_sleepHours_steps`).

Section 2: API Architecture

Q6. What is the overall API architecture?

A: Microservices architecture with 4 separate FastAPI servers:

Service	Purpose
stress_backend	Auth, routing, history, orchestration (API Gateway)
stress_api	ML stress prediction model
blink_api	Eye blink detection from video
stress_chatbot_api	AI health chatbot (Groq/LLaMA)

The Android app only talks to stress_backend, which proxies to other services.

Q7. What is the complete API endpoint list?

Method	Endpoint	Auth	Purpose
GET	/	No	Health check
POST	/signup	No	User registration + OTP
POST	/verify_email	No	OTP verification
POST	/login	No	User login
POST	/predict_stress	JWT	Submit scan for prediction
GET	/history/{user_id}	JWT	Get user's scan history
POST	/chat	No	Chatbot proxy
POST	/change_password	JWT	Change password
PUT	/update_profile	JWT	Update user profile

Q8. How does the Android app communicate with the backend?

A: Using Retrofit 2 with OkHttp and Gson. Three separate OkHttp clients: general (60s timeout, 2 retries), auth (120s, no retries — prevents duplicate signups), and blink (90s, 1 retry for video upload).

Q9. How does the stress prediction API call chain work?



Android sends StressInput JSON with Bearer token. Backend verifies JWT, calls stress_api with 3 retries (2s delay). Stress API processes input through model, returns prediction (0/1/2). Backend maps to "Low"/"Medium"/"High", saves to history, returns result.

Q10. How does the Blink API work?

A: Android records a 10-second face video via CameraX, uploads as multipart. Server uses **MediaPipe FaceMesh** (468 landmarks). **EAR formula:** $EAR = (A + B) / (2.0 \times C)$ where A,B = vertical eye distances, C = horizontal. Blink detected when $EAR < 0.21$ for ≥ 2 consecutive frames.

Q11. How does the Chatbot API work?

A: Uses **Groq API** with **LLaMA 3.3 70B** (temperature=0.4). Three-layer filtering: emergency detection (critical phrases → emergency response), medical topic filter (rejects off-topic), and system prompt (wellness advice only, no diagnosis). Appends medical disclaimer.

Section 3: Authentication & Login

Q12. How does user registration (signup) work?

A: Two-flow system:

- **OTP Flow** (email service works): Generate 6-digit OTP → store in `pending_users` → send email → user verifies → move to `users` → issue JWT
- **Direct Flow** (SMTP fails on Render): Create account directly → issue JWT immediately

Q13. How does OTP email sending work?

A: Dual provider with automatic failover: **Primary** — Resend API (HTTP POST, works on Render). **Fallback** — Gmail SMTP (SSL port 465, works locally). If both fail → account created without verification.

Q14. How does JWT authentication work?

A: Algorithm: **HS256**. Expiry: 30 days (43,200 minutes). Token contains `{"sub": user_id, "exp": expiry}`. Created with `python-jose`. Verified via FastAPI dependency `Depends(verify_token)`. Android stores token in DataStore, sends as `Authorization: Bearer <token>` header.

Q15. How is password security handled?

A: **bcrypt** via `passlib.CryptContext`. Input truncated to 72 chars (bcrypt max). Change password verifies old password first, then updates hash in MongoDB.

Q16. How does Android manage user sessions?

A: Jetpack **DataStore Preferences** stores token, user ID, login flag, name, email, phone. All exposed as Kotlin `Flow<T>` for reactive UI. On logout: only clears auth token — preserves profile for re-login. Name fallback: backend → saved name → email prefix.

Section 4: Parameter Linking & Data Flow

Q17. What are the 12 input parameters for stress prediction?

#	Parameter	Type	Source on Android
1	EyeBlinkRate_per_min	float	Blink API (video)

2	Resting_Heart_Rate_BPM	float	Camera PPG scan
3	Heart_Rate_After_Exercise_BPM	float	Camera PPG scan
4	age	int	Manual input
5	gender	string	Dropdown selection
6	worklife	string	Dropdown selection
7	Sleep_Duration_Hours	float	Auto (sensors) or manual
8	Chest_Pain	int 0/1/2	Dropdown selection
9	Cholesterol_mg_dL	float	Manual input
10	ECG_Result	int 0/1/2	Dropdown selection
11	Body_Temperature_C	float	Manual input
12	Steps_per_Day	int	Auto (pedometer) or manual

Q18. How are categorical parameters encoded between Android and ML model?

A: Multi-stage mapping:

1. **Android → Backend:** Strings to integers (Chest_Pain: "No"→0, "Mild"→1, "Yes"→2; ECG: "Normal"→0, "Sinus_Tachycardia"→2)
2. **Backend → Stress API:** Integers back to strings (0→"No", 1→"Mild", 2→"Yes")
3. **Stress API → Model:** `LabelEncoder.transform()` converts strings to model-encoded integers

Q19. How does `@SerializedName` link Android fields to backend?

```
@SerializedName("Eye_Blink_Rate_per_min") val eyeBlinkRate: Int
@SerializedName("Resting_Heart_Rate_BPM") val restingHeartRate: Int
@SerializedName("Sleep_Duration_Hours") val sleepDuration: Double
```

For history, alternate arrays handle multiple naming conventions across backend versions.

Q20. How does history JSON parsing work for varying schemas?

A: Recursive JSON flattener: `flattenJson()` walks the JSON creating a flat map with dotted paths. Handles MongoDB Extended JSON (`$date`, `$oid`, `$numberInt`). Finder functions search with multiple key aliases and case-insensitive matching — resilient to backend schema changes.

Section 5: ML Model & Stress Prediction

Q21. What ML algorithm is used?

A: Random Forest Classifier: 200 trees, max_depth=15, min_samples_split=5, min_samples_leaf=2, class_weight='balanced'.
3-class output: Low(0), Medium(1), High(2).

Q22. How is the ML pipeline structured?

A: 1) `LabelEncoder` for categoricals (gender, worklife, Chest_Pain, ECG_Result). 2) `StandardScaler` normalizes all 12 features.
3) `model.predict()` returns class. All serialized with `joblib` as `.pkl` files.

Q23. Why does feature order matter?

A: Feature order must match exactly between training and inference. The scaler and model expect values at specific positions.
`retrain_model.py` verifies alignment with the API's `feature_order` array. Mismatch produces incorrect predictions.

Section 6: Sleep Detection

Q24. How does the sleep detection state machine work?



- **AWAKE → POSSIBLY_SLEEPING:** Screen off ≥ 15 min AND confidence ≥ 40
- **POSSIBLY_SLEEPING → SLEEPING:** Screen off ≥ 30 min AND still AND confidence ≥ 60
- **SLEEPING → AWAKE:** Real unlock (not brief); records session if ≥ 1 hour
- Brief unlocks (< 2 min) are ignored

Evaluation loop runs every 60 seconds for battery efficiency.

Q25. How is sleep confidence calculated?

Signal	Weight	Criteria
Screen Off	30	Full at ≥30min, 2/3 at ≥15min, 1/3 at ≥10min
Motion Still	30	Full at ≥30min, 2/3 at ≥20min, 1/3 at ≥10min
Sleep Window	25	Full if in predicted sleep hours
No Unlock	15	Full if no unlock for ≥15min

Q26. What is the sleep data fallback hierarchy?

1. **SleepDetectionManager** — real-time sensor-based (screen + accelerometer)
2. **SleepPatternRepository** — historical sessions (last 14 days, SharedPreferences)
3. **HealthConnectManager** — Health Connect API → UsageStats gap analysis → time-based estimate

Q27. How does sleep window prediction use circular mean?

```
angle = hour * 2π / 24  
mean = atan2(Σsin/n, Σcos/n)  
predictedHour = mean * 24 / 2π
```

Handles midnight wrap-around. Defaults: Day shift 9PM–8AM, Night shift 6AM–4PM. Requires ≥3 sessions.

Section 7: Step Counting

Q28. How does step counting work on Android?

A: Three-tier sensor priority:

1. **TYPE_STEP_COUNTER** (preferred): Cumulative since boot. Daily = current – baseSteps.
2. **TYPE_STEP_DETECTOR** (backup): Event-based, increments per step.
3. **TYPE_ACCELEROMETER** (fallback): Peak detection — magnitude crosses threshold (11.5) upward with ≥300ms debounce.

Q29. How does background step counting work?

A: `StepCountService` runs as a foreground service with persistent notification, saves daily steps to `SharedPreferences`.

`StepManager` reads via `getBackgroundSteps()` when app opens.

Section 8: Blink Detection (EAR Algorithm)

Q30. What is the Eye Aspect Ratio (EAR) formula?

```
EAR = (|P2-P6| + |P3-P5|) / (2 * |P1-P4|)
```

MediaPipe landmarks — Left eye: [33,160,158,133,153,144], Right eye: [362,385,387,263,373,380]. Final EAR = average of both eyes.

Q31. How is a blink detected from EAR values?

A: Threshold: $EAR < 0.21$ = closed eyes. Must stay below for ≥2 consecutive frames. Blink counted when EAR rises back above threshold. Prevents false positives from single-frame noise.

Section 9: AI Integration (Gemini & Groq)

Q32. How does Gemini wellness tips generation work?

A: After prediction, all 12 parameters + stress level sent to **Gemini 2.0 Flash** (temperature=0.7, maxTokens=400). Prompt identifies problems and generates 2–4 actionable tips. Fallback: rule-based `defaultTips()` from user's actual data if API fails.

Q33. How does the chatbot's safety filtering work?

A: Three layers: 1) Emergency detection → immediate response. 2) Medical topic filter → rejects off-topic. 3) System prompt → wellness advice only, no diagnoses or prescriptions.

Section 10: Deployment & Infrastructure

Q34. Where are services deployed and how are cold starts handled?

A: All 4 services on **Render.com** free tier (spins down after 15 min). Mitigation: `wakeUpBackend()` pings before auth calls, `preWarmBlinkApi()` on assessment screen load, auth timeout extended to 120s, retry interceptor with 2s delay.

Section 11: Security

Q35. How are API routes protected?

A: FastAPI's `Depends(verify_token)` extracts Bearer token, decodes with `python-jose`, raises `HTTPException(401)` if invalid/expired.

Q36. What are the security concerns?

- JWT secret hardcoded ("mysecretkey123") — should be in env vars
- No rate limiting on auth endpoints
- CORS not configured on main backend
- No input range validation on health parameters

Section 12: Local Storage (Android)

Q37. What local storage mechanisms are used?

Storage	Technology	Data
User session	Jetpack DataStore	Token, user ID, name, email, phone, shift type
Scan timestamps	SharedPreferences	Local timestamps matched to backend history
Step data	SharedPreferences	Base steps, daily steps, accel steps
Sleep sessions	SharedPreferences	Last 14 sessions (JSON/Gson)
Hidden scans	SharedPreferences	Fingerprints of "deleted" scans

Q38. How does scan deletion work without backend support?

A: Client-side hiding: fingerprint generated from all params + result → stored in `StringSet` → filtered out on load. Data persists on backend but hidden on device.

Section 13: Motion Sensor

Q39. How does motion stillness detection work?

A: Accelerometer at `SENSOR_DELAY_NORMAL` with 5s batching. Deviation from gravity: $|\sqrt{x^2+y^2+z^2} - 9.81|$. Rolling window of 60 samples (~5 min). Stillness: avg deviation < 0.5 for ≥ 20 min. Motion spike: was still 20+ min, current deviation > 1.0.

Section 14: Architecture & Design Patterns

Q40. What is the Android app architecture?

A: MVVM — View: Jetpack Compose screens. ViewModel: AuthViewModel, AssessmentViewModel, CameraScanViewModel. Repository: single orchestrator. Data layer: UserPreferences, HealthConnectManager, StepManager, SleepDetectionManager. Network: Retrofit + OkHttp.

Q41. How is dependency injection handled?

A: Manual ViewModelProvider.Factory pattern (no Hilt/Dagger). MindEaseApp Application class initializes shared instances.

Q42. How does the multi-step assessment flow work?

A: 6-step wizard: 1) Basic info. 2) Resting HR (camera PPG). 3) Blink rate (video → API). 4) Post-exercise HR. 5) Sleep + steps (auto/manual). 6) Review → prediction + AI tips. State managed via AssessmentState data class with MutableStateFlow.